Course Title: AI Assisted Coding

Assignment Number: 21.2

Name : P. Samanvith
Batch No : 40
HT No : 2303A52090

Task 1 – AI-Assisted Pair Programming

Task Description

Use AI to generate an initial program (a simple To-Do List Manager), ask AI to suggest enhancements (error handling, additional features), and compare the original and improved versions.

Part A: Initial Program

Prompt: Write a simple, basic command-line To-Do List Manager in Python.

Code:

```
# Initial To-Do List Manager
todo_list = []

def add_task(task):
    todo_list.append(task)
    print(f"Task '{task}' added.")

def view_tasks():
    print("\n--- To-Do List ---")
    for index, task in enumerate(todo_list):
        print(f"{index + 1}. {task}")

# Basic usage
add_task("Read Python basics")
add_task("Complete Lab 21")
view_tasks()
```

Output:

```
Task 'Read Python basics' added.
Task 'Complete Lab 21' added.
```

```
--- To-Do List ---
1. Read Python basics
2. Complete Lab 21
```

Part B: AI-Suggested Enhancements

Prompt: Enhance the previous To-Do List Manager by adding error handling, a feature to mark tasks as completed, and input validation.

Code:

```
# Enhanced To-Do List Manager
todo_list = []

def add_task(task):
    if not task.strip():
        print("Error: Task description cannot be empty.")
        return
    todo_list.append({"task": task, "completed": False})
    print(f"Success: Task '{task}' added.")

def view_tasks():
    print("\n--- To-Do List ---")
    if not todo_list:
        print("Your list is currently empty.")
        return

    for index, item in enumerate(todo_list):
        status = "[X]" if item["completed"] else "[ ]"
        print(f"{index + 1}. {status} {item['task']}")

def complete_task(index):
    try:
        task_idx = int(index) - 1
        if task_idx < 0 or task_idx >= len(todo_list):
            print("Error: Invalid task number.")
            return
        todo_list[task_idx]["completed"] = True
        print(f"Success: Task '{todo_list[task_idx]['task']}' marked as completed.")
    except ValueError:
        print("Error: Please enter a valid number.")

# Enhanced usage
```

```
add_task("Write Lab Report")
add_task("") # Testing error handling
view_tasks()
complete_task(1)
complete_task(5) # Testing out of bounds error
view_tasks()
```

Output:

```
Success: Task 'Write Lab Report' added.
Error: Task description cannot be empty.

--- To-Do List ---
1. [ ] Write Lab Report
Success: Task 'Write Lab Report' marked as completed.
Error: Invalid task number.

--- To-Do List ---
1. [X] Write Lab Report
```

Comparison & Reflection

Role of AI in Collaboration: The AI acted as an efficient pair programmer. The initial version was a bare-bones script utilizing a simple list of strings. Through AI collaboration, the data structure was refactored into a list of dictionaries to track the "completed" state. AI seamlessly introduced try-except blocks and input validation (strip()), demonstrating how AI can rapidly mature prototype code into robust, production-ready logic.

Task 2 – Version Control Integration

Task Description

Initialize a Git repository, create a branch, add AI-assisted modifications, commit with a meaningful message, merge back into the main branch, and reflect on the workflow.

Prompt

Provide the exact Git bash commands required to initialize a repository, branch out to add the enhanced To-Do list features, commit the changes, and merge them back into the main branch.

Code (Git Commands)

```
# 1. Initialize a Git repository
git init
```

```
# 2. Add the initial version of the code and commit
git add todo.py
git commit -m "Initial commit: Basic To-Do List manager"

# 3. Create and switch to a new branch for enhancements
git checkout -b feature-ai-enhancements

# (At this point, we replace the contents of todo.py with the enhanced code)

# 4. Add the AI-assisted modifications
git add todo.py

# 5. Commit with a meaningful message
git commit -m "feat: implement error handling and task completion status"

# 6. Switch back to the main branch
git checkout main

# 7. Merge changes back into the main branch
git merge feature-ai-enhancements
```

Output (Simulated Terminal Output)

```
Initialized empty Git repository in /project/.git/
[main (root-commit) 1a2b3c4] Initial commit: Basic To-Do List manager
 1 file changed, 14 insertions(+)
 create mode 100644 todo.py
Switched to a new branch 'feature-ai-enhancements'
[feature-ai-enhancements 5d6e7f8] feat: implement error handling and task completion status
 1 file changed, 25 insertions(+), 5 deletions(-)
Switched to branch 'main'
Updating 1a2b3c4..5d6e7f8
Fast-forward
 todo.py | 30 ++++++++++++++++++++++++-----
 1 file changed, 25 insertions(+), 5 deletions(-)
```

Reflection

Version Control & AI Collaboration: Version control acts as a safety net when experimenting with AI-generated code. Because AI can sometimes hallucinate or introduce bugs, creating a dedicated branch (feature-ai-enhancements) isolates the AI's suggestions. Once the AI code is tested and verified, merging it into main ensures the codebase remains stable.

Task 3 – AI for Commit Message Generation

Task Description

Use AI to generate concise, meaningful commit messages based on the changes made, compare them with human-written ones, and discuss documentation improvements.

Prompt

Generate 3 options for a conventional commit message based on the following changes: Changed the To-Do list data structure to a dictionary to track completed status, added try-except blocks for invalid inputs, and added a complete_task function.

Output

AI-Generated Commit Messages:

1. feat(todo): add task completion tracking and input validation
2. refactor: update data structure for task status and enhance error handling
3. feat: implement complete_task function with try-except safety blocks

Human-Written Equivalent (Typical Example):

Updated todo list code to work better and added complete feature

Discussion

Documentation and Traceability: The AI-generated messages strictly adhere to "Conventional Commits" standards (using prefixes like feat and refactor). They explicitly state *what* changed and *why* (e.g., "tracking and input validation"). The human-written example is vague. Using AI for commit messages greatly improves Git history traceability, making it easier for future developers to understand the project's evolution without reading every line of code diffs.

Task 4 – Pair Programming Simulation

Task Description

Simulate a workflow where Student A writes an initial program, Student B (with AI) improves it, and both collaborate using Git.

Simulation Workflow

Step 1: Student A writes the initial program (Main Branch)

Student A creates a simple math_utils.py script.

```
# math_utils.py (Student A)
def add(a, b):
    return a + b
```

```
print(add(5, 3))
```

Student A Git Commands:

```
git init
git add math_utils.py
git commit -m "init: add basic addition function"
```

Step 2: Student B uses AI to suggest improvements (New Branch)

Student B pulls the code, creates a branch, and prompts AI: "Add subtraction, multiplication, and safe division to this math script."

Code (Student B with AI Assistance):

```
# math_utils.py (Student B + AI)
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    if b == 0:
        return "Error: Cannot divide by zero"
    return a / b

# Test cases
print(add(5, 3))
print(divide(10, 0))
```

Step 3: Student B Commits Changes

Student B Git Commands:

```
git checkout -b student-b-features
git add math_utils.py
git commit -m "feat(math): add sub, mult, and safe div functions"
```

Step 4: Collaborative Merge

Students A and B review the code together, ensuring the divide-by-zero check works. They then merge it.

```
git checkout main
git merge student-b-features
git branch -d student-b-features
```

Output / Conclusion

By simulating this pair programming exercise, it's evident that AI significantly speeds up the feature-addition phase for Student B. The use of Git branching ensures that Student A's original working code is never overwritten until Student B's AI-generated additions are fully reviewed and validated.